

Meta Learning

Prototypical, Relation Network, and MAML

Praveen Kumar Chandaliya, PhD

pkc@aid.svnit.ac.in

Department of Artificial Intelligence

Sardar Vallabhbhai National Institute of Technology , Surat



<https://acesse.one/8ej1sp7>

Introduction to Few-Shot Learning

Supervised learning is very prone to overfitting: CIFAR-10, we force the model to optimize for 10 classes of images, but if we ask the model about the 11th class, and it will be clueless.

We cannot train a classifier using conventional methods here; any modern classification algorithm **will depend on far more parameters** than there are training examples and will **generalize poorly**. If the data is insufficient to constrain the problem, one possible solution is to achieve few-shot learning, which falls under the umbrella of meta-learning.

What is Few-shot Learning?

Few-shot learning refers to the ability of machine learning models to learn new concepts with only a few examples. It can be very helpful in cases where:

- One wants to avoid data hunger due to the high resource and computation cost of training a model with a large amount of data.
- It's nearly impossible to have access to large amounts of labelled data due to the nature of the problem or privacy concerns.
- The solution will be used in scenarios where the model needs to adapt quickly to new tasks or domains before a large amount of labelled data is collected.

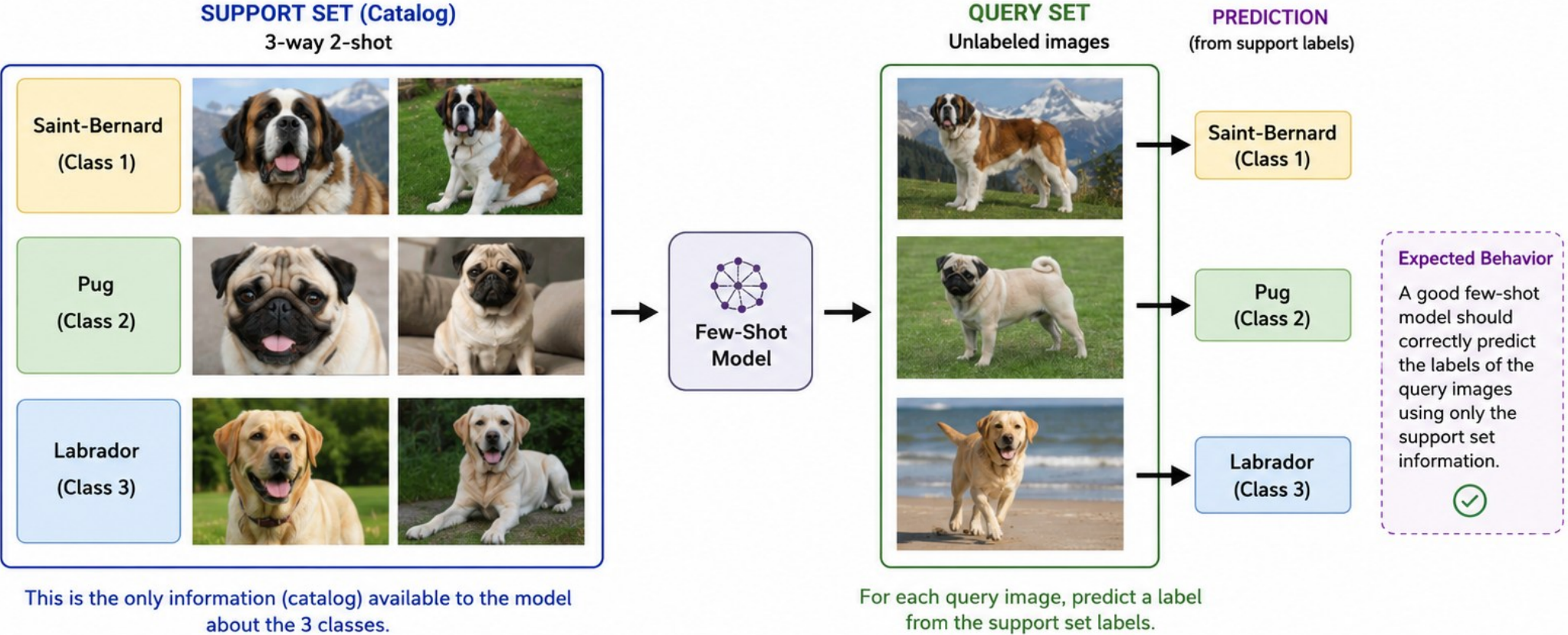
Few-Shot Classification: 3-Way 2-Shot Example

In a few-shot classification task, we have a labeled support set (which acts like a catalog) and a query set. For each image in the query set, we want to predict a label from the labels present in the support set. A few-shot classification model must use the information from the support set to classify query images. We say few-shot when the support set contains very few images for each label (typically less than 10). The figure below shows a 3-way 2-shots classification task.

- **3-way** means **3 different classes**.
- **2-shots** means **2 examples per class**.

💡 The model has never seen any Saint-Bernard, Pug or Labrador during its training.

The support set is the only information the model has about these classes.



Meta-learning framework for few-shot learning

In the classical learning framework, we learn how to classify from training data and evaluate the results using test data. In the meta-learning framework, we *learn how to learn* to classify, given a **set of training tasks** and **evaluate on a set of test tasks**; in other words, we use one set of classification problems to help solve other, unrelated sets.

Training task 1

Support set



$K=2$

$N=3$

Query set



Training task 2 . . .

Support set

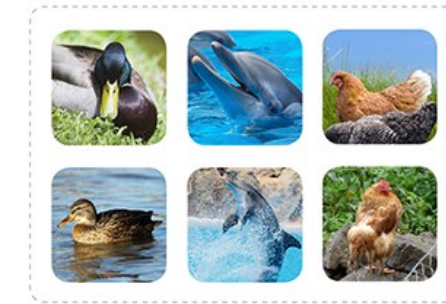


Query set



Test task 1 . . .

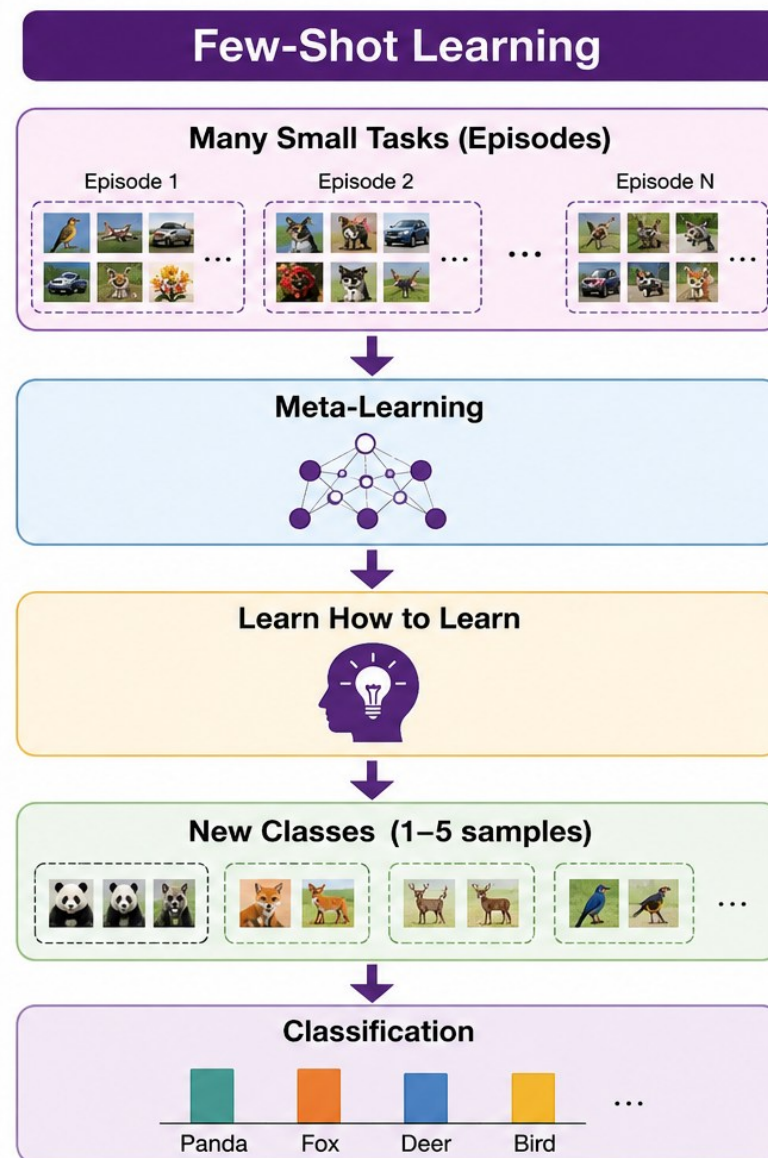
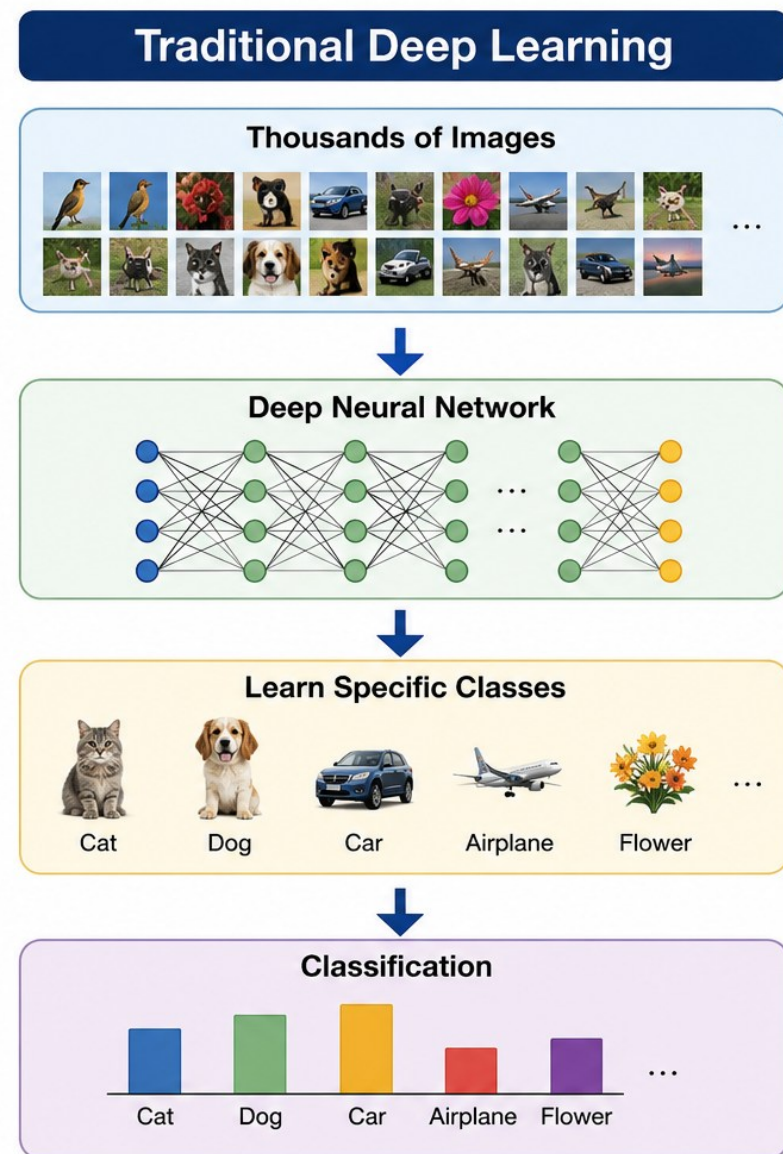
Support set



Query set



Traditional Deep Learning vs Few-Shot Learning



Colab

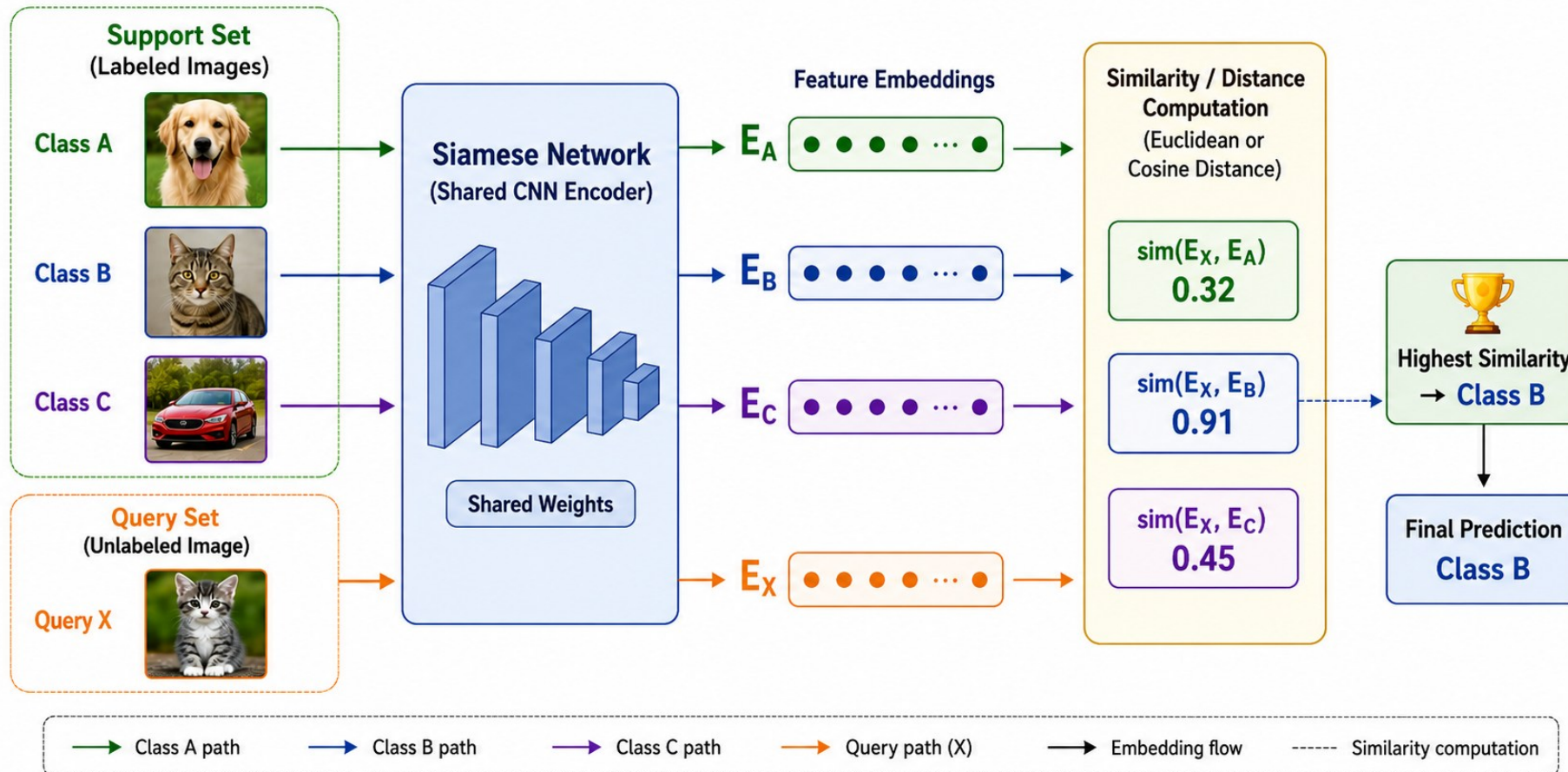
FewShot_Classification_Omniglot.ipynb

https://github.com/praveenkumarchandaliya/STTP_Program/tree/MetaLearning

Siamese Networks Few Shot Learning

Siamese Networks solve Few-Shot Learning (FSL) by evaluating **similarity** rather than direct classification. Instead of teaching a model to recognize an object in isolation, the network learns to determine whether two inputs belong to the same class.

SIAMESE NETWORK IN FEW-SHOT LEARNING



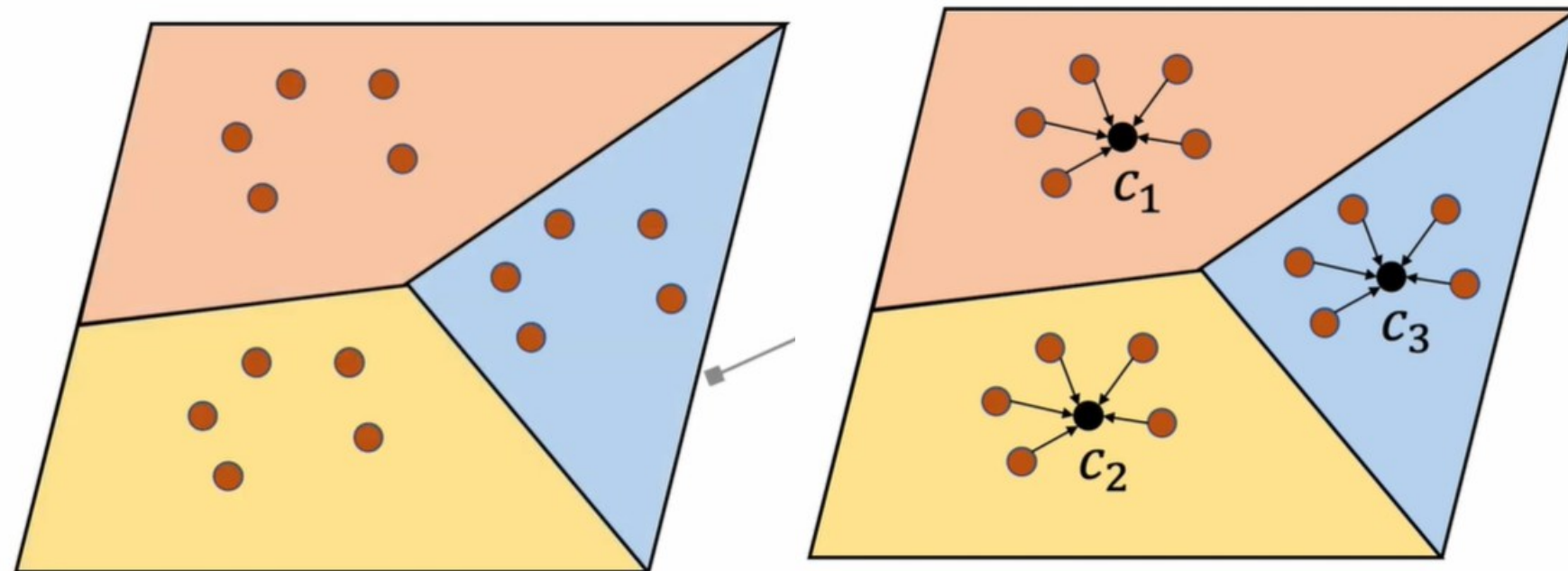
Prototypical Network

Prototypical Network (ProtoNet) is one of the most influential and widely used **metric-based few-shot learning** approach proposed by Jake Snell and colleagues in 2017.

Instead of learning a classifier, ProtoNet learns an **embedding space** in which:

- Samples from the same class are close together.
- Samples from different classes are far apart.

Each class is represented by a **prototype**, which is the mean feature vector of the support samples belonging to that class.



3-ways 5-shot

Prototype Computation

For class c ,

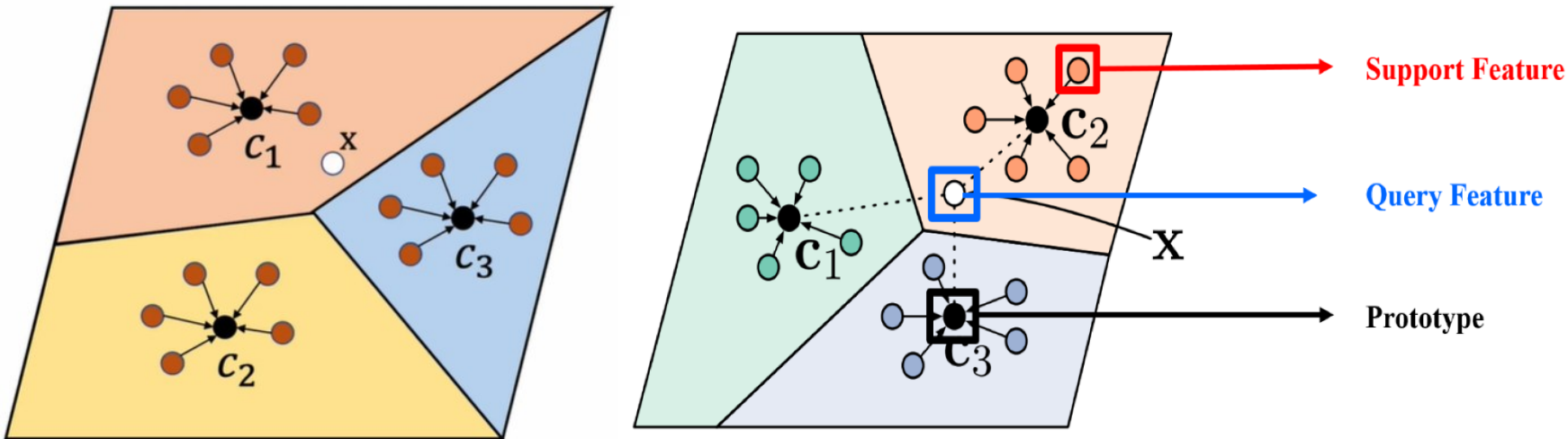
$$p_c = \frac{1}{K} \sum_{i: y_i=c} f_{\theta}(x_i)$$

where

- K : number of support examples
- f_{θ} : embedding network

Prototypical Networks (Snell et al., NeurIPS 2017)

Prototypical Network



Query Classification

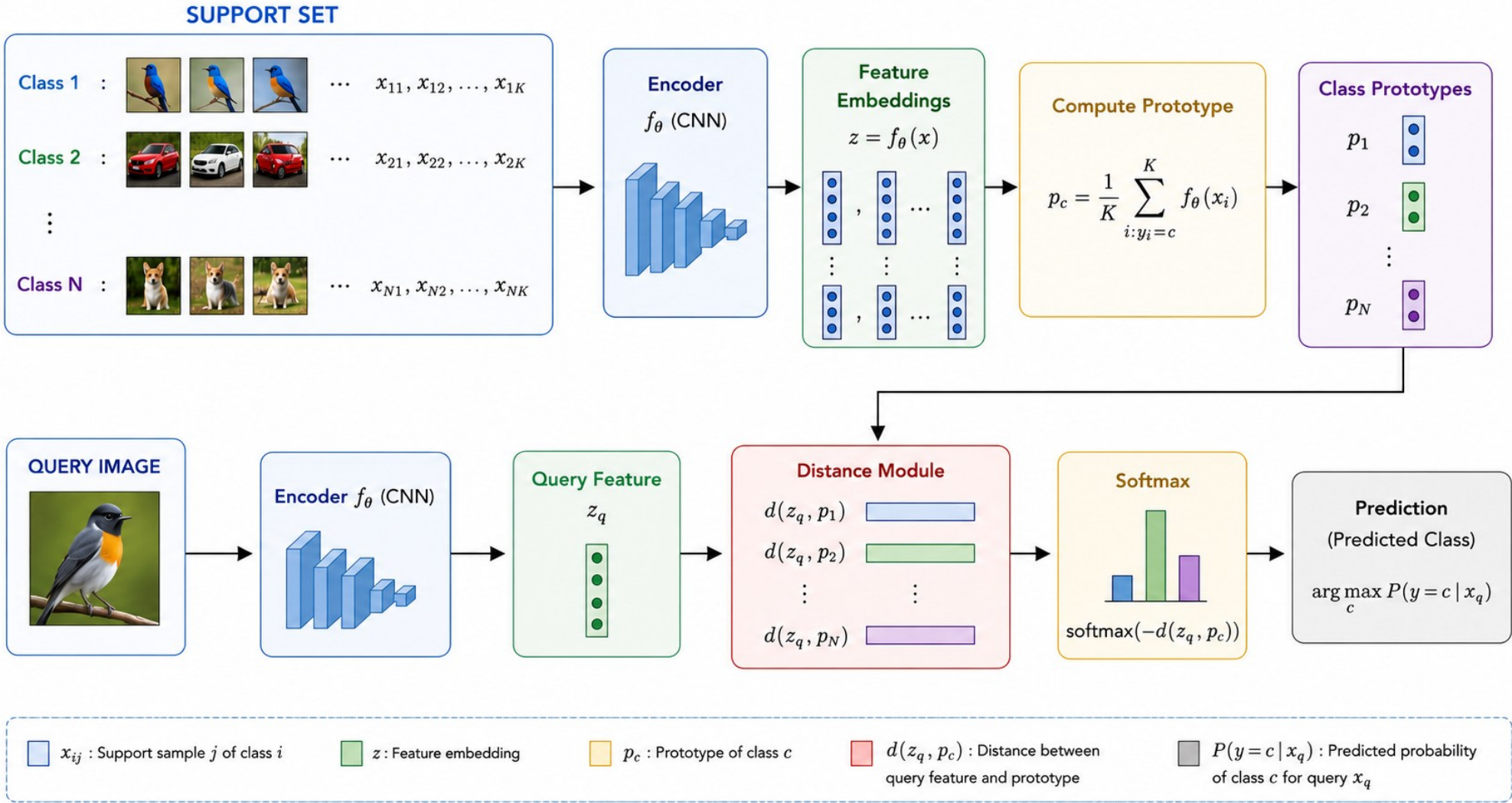
Compute Euclidean distance:

$$d(x_q, p_c) = \|f_\theta(x_q) - p_c\|^2$$

Probability:

$$P(y = c | x_q) = \frac{e^{-d(x_q, p_c)}}{\sum_{c'} e^{-d(x_q, p_{c'})}}$$

Prototypical Network



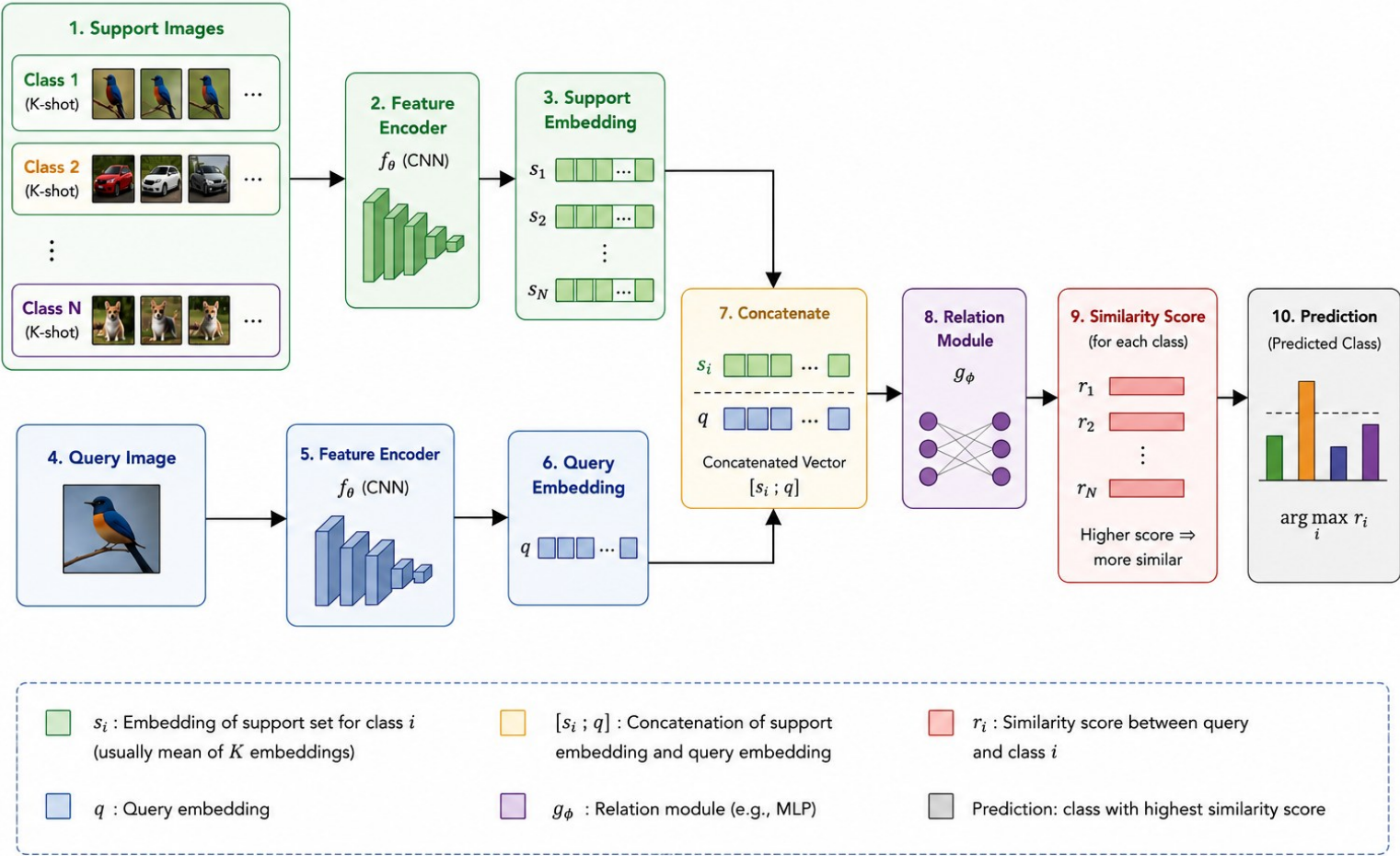
Characteristics: Simple, Fast, Few parameters, Strong baseline

Source: Generate Figure using ChatGPT

Prototype and Relation_Example_Math

Relation Network (Relation Net)

Relation Network (RelationNet) is a **metric-based few-shot learning** method proposed by Flood Sung et al. 2018. Unlike **Prototypical Networks**, which use Euclidean distance, RelationNet **learns a deep nonlinear similarity function** between support and query samples.



Relation Model:

- MLP-Based Relation Network
- CNN-Based Relation Module
- Attention-Based Relation Network
- Graph Relation Network

Relation Score

For class c ,

$$r_c = g_\phi([f_\theta(x_q), p_c])$$

where

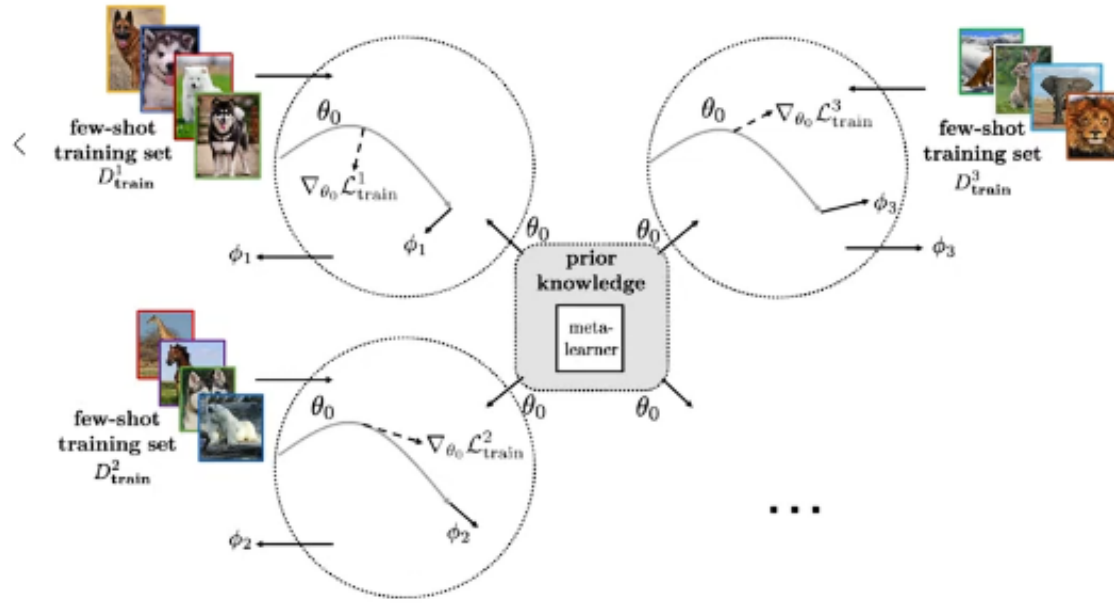
- f_θ : feature extractor
- g_ϕ : relation network

Prototype and Relation_Example_Math

Model Agnostic Meta Learning

Parameter Initialization Method

Learning to Fine-Tune aims to learn a parameter initialization θ that can quickly adapt to new tasks.



Method:

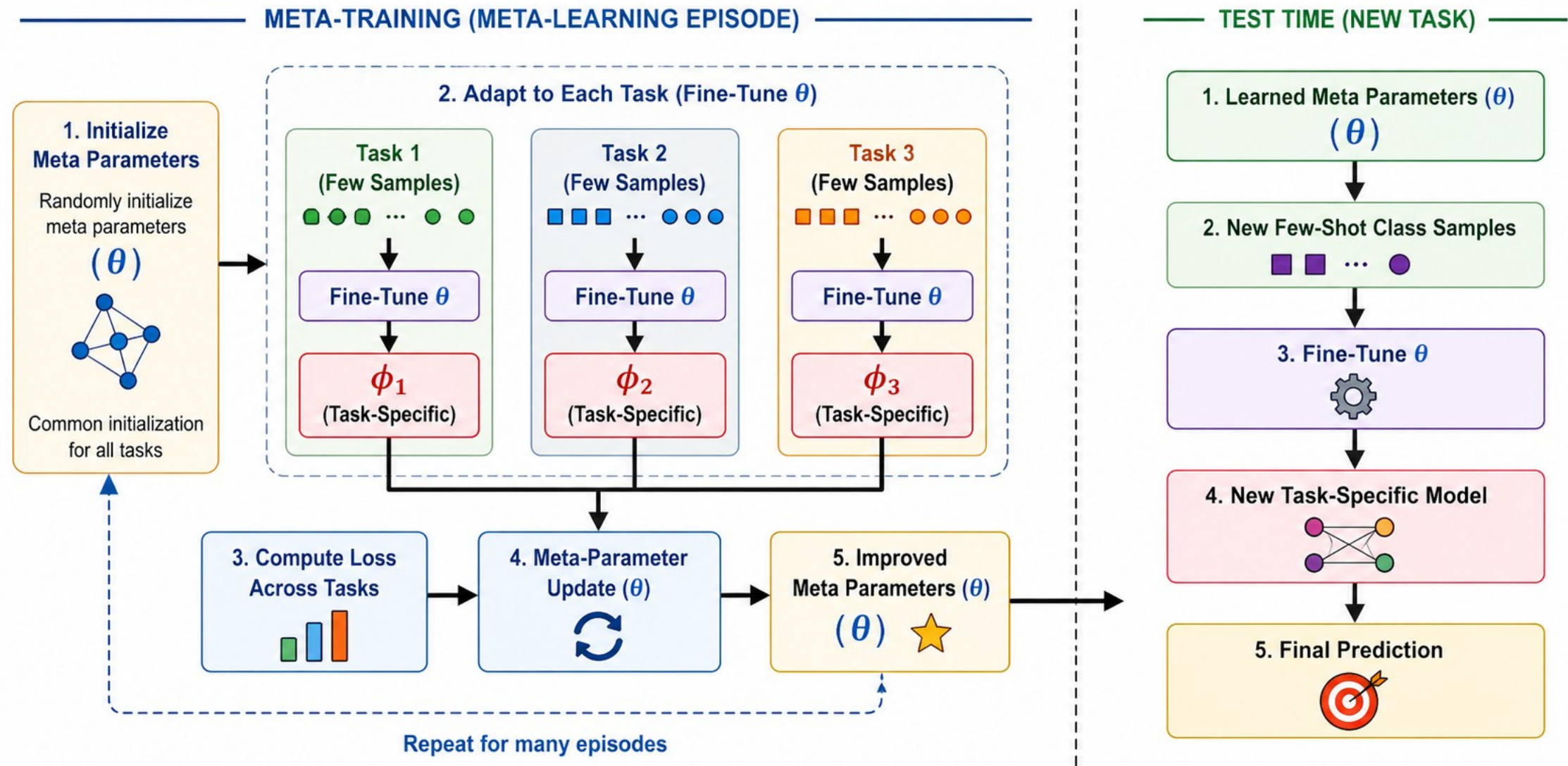
- For each task/episode $(D_{train}^i, D_{test}^i)$, update task-specific parameters Φ_i to minimize $L(\theta, D_{train}^i)$
- Update meta parameter θ to minimize $\sum L(\Phi_i, D_{test}^i)$
- At test time, use few gradient steps to adapt classify novel classes

Θ : All initialize parameters for the class (task) (Start with a common parameter set)

Φ_i : model parameter of a specific task (Φ_1, Φ_2, Φ_3 , model parameter for specific task1, task2, task3)

$\sum L()$: Weighted sum of each of these losses corresponding to each of those tasks.

LEARNING TO FINE-TUNE



Parameter Initialization Method

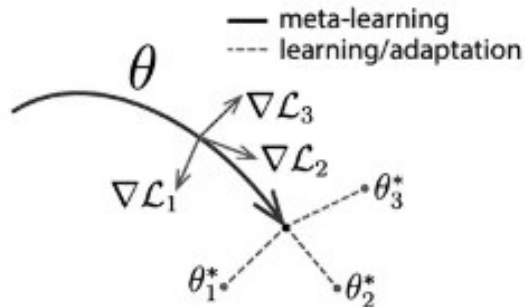
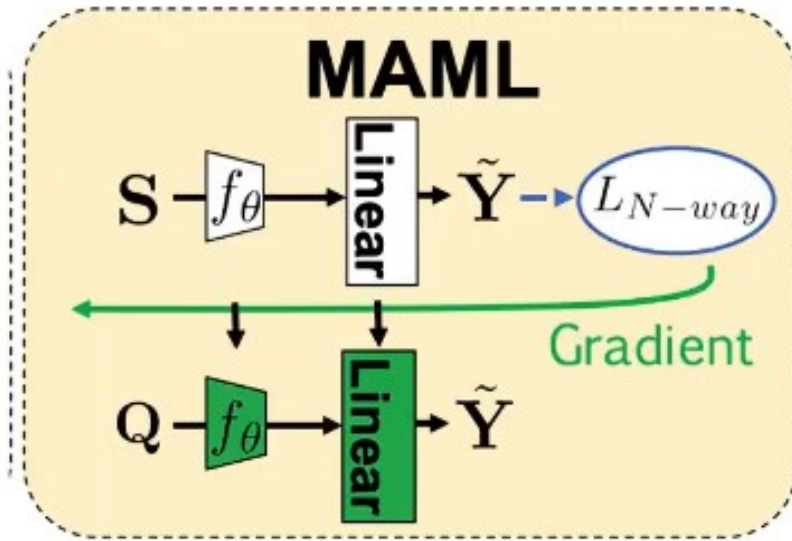


Figure 1. Diagram of our model-agnostic meta-learning algorithm (MAML), which optimizes for a representation θ that can quickly adapt to new tasks.

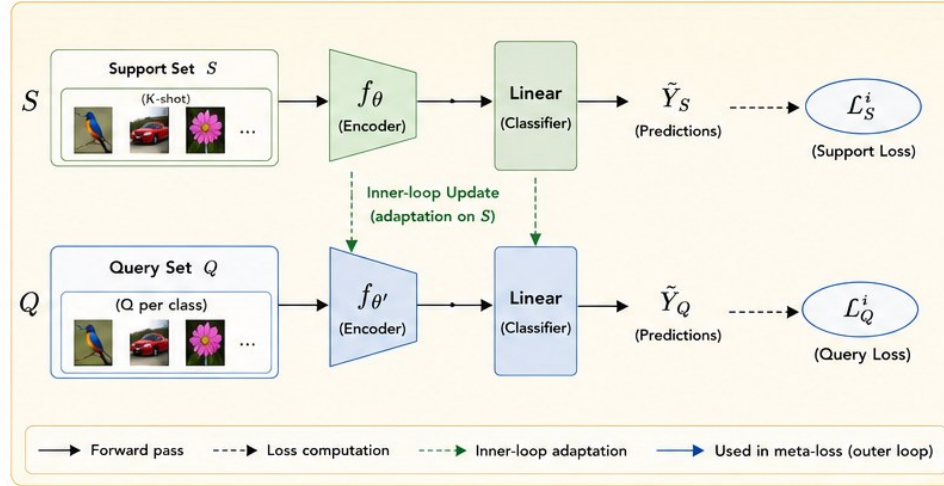
Key Idea: Acquire task-specific parameters (Φ_i) through optimization

Formulation

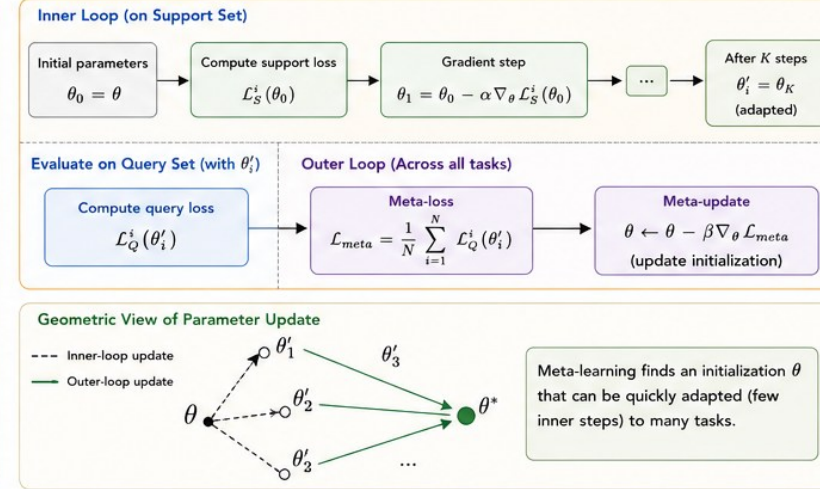
- Learn prior such that model can adapt to new tasks
One form of prior knowledge: **Parameter Initialization**
- Task-specific Update:**
 $\theta'_i = \theta - \alpha \nabla_{\theta} L(\theta, D_{train}^i)$ (Single or multiple SGD updates)
- Meta Fine-tuning:**
 $\theta = \theta - \beta \nabla_{\theta} \sum_i L(\theta'_i, D_{test}^i)$
 $\theta = \theta - \beta \nabla_{\theta} \sum_i L(\theta - \alpha \nabla_{\theta} L(\theta, D_{train}^i), D_{test}^i)$ (Second-order derivatives)

Parameter Initialization Method

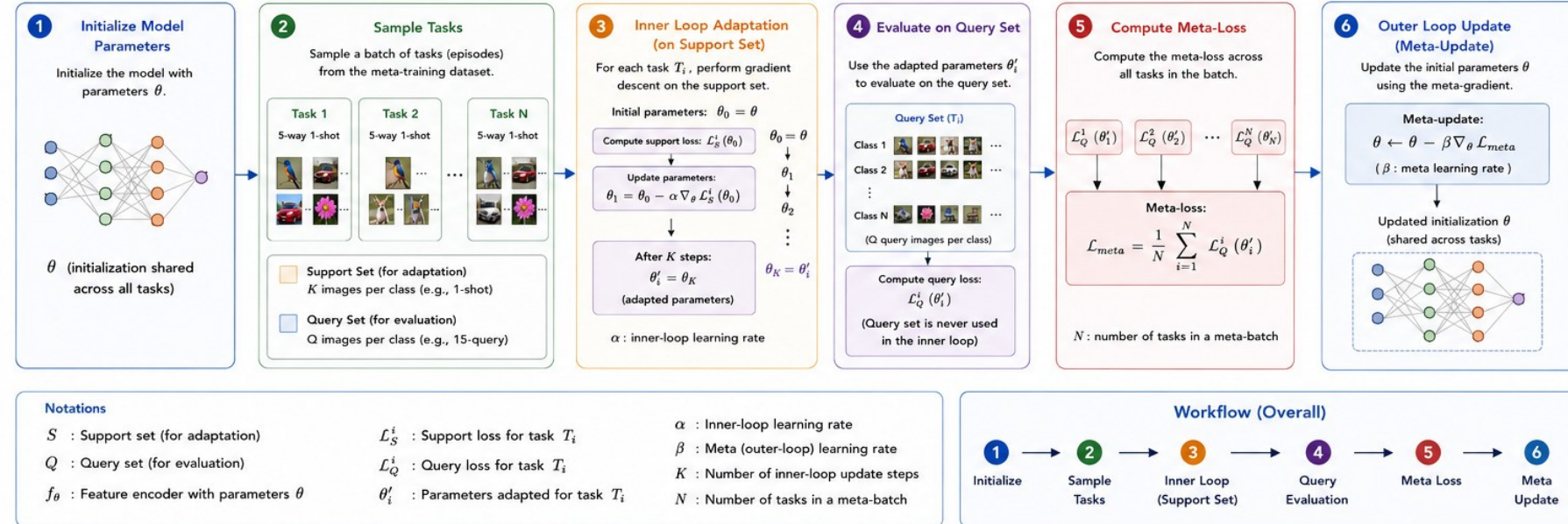
MAML MODEL ARCHITECTURE



PARAMETER UPDATE IN MAML (PER TASK)



MAML TRAINING: STEP-BY-STEP (1 TO 6)



MAML_Step_by_Step

MAML for few-shot Supervised Learning

Algorithm 2 MAML for Few-Shot Supervised Learning

Require: $p(\mathcal{T})$: distribution over tasks

Require: α, β : step size hyperparameters

```
1: randomly initialize  $\theta$ 
2: while not done do
3:   Sample batch of tasks  $\mathcal{T}_i \sim p(\mathcal{T})$ 
4:   for all  $\mathcal{T}_i$  do
5:     Sample  $K$  datapoints  $\mathcal{D} = \{\mathbf{x}^{(j)}, \mathbf{y}^{(j)}\}$  from  $\mathcal{T}_i$ 
6:     Evaluate  $\nabla_{\theta} \mathcal{L}_{\mathcal{T}_i}(f_{\theta})$  using  $\mathcal{D}$  and  $\mathcal{L}_{\mathcal{T}_i}$  in Equation (2)
       or (3)
7:     Compute adapted parameters with gradient descent:
        $\theta'_i = \theta - \alpha \nabla_{\theta} \mathcal{L}_{\mathcal{T}_i}(f_{\theta})$ 
8:     Sample datapoints  $\mathcal{D}'_i = \{\mathbf{x}^{(j)}, \mathbf{y}^{(j)}\}$  from  $\mathcal{T}_i$  for the
       meta-update
9:   end for
10:  Update  $\theta \leftarrow \theta - \beta \nabla_{\theta} \sum_{\mathcal{T}_i \sim p(\mathcal{T})} \mathcal{L}_{\mathcal{T}_i}(f_{\theta'_i})$  using each  $\mathcal{D}'_i$ 
    and  $\mathcal{L}_{\mathcal{T}_i}$  in Equation 2 or 3
11: end while
```

Initialize θ

for iteration in range(meta_iterations):

 sample batch of tasks

 meta_loss = 0

for task in tasks:

 support_set, query_set = split(task)

 # Inner loop

 loss_support = compute_loss(θ , support_set)

$\theta_{\text{prime}} = \theta - \alpha * \text{grad}(\text{loss_support})$

 # Query loss

 loss_query = compute_loss(θ_{prime} , query_set)

 meta_loss += loss_query

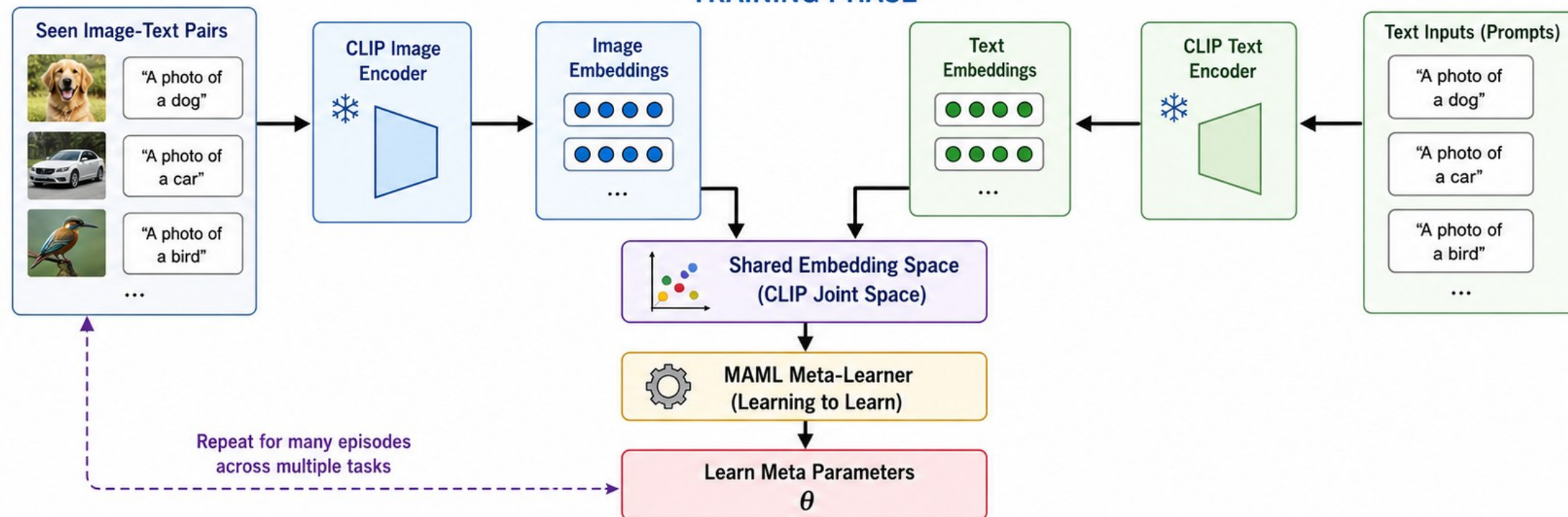
 # Outer loop

$\theta = \theta - \beta * \text{grad}(\text{meta_loss})$

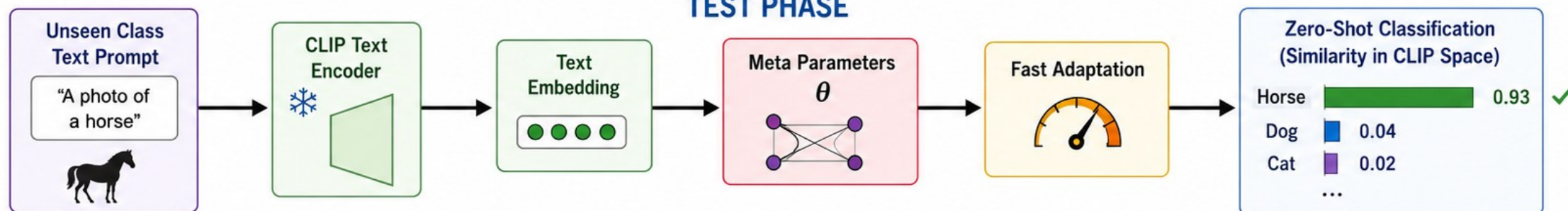
- MAML does not learn a single task. It learns how to quickly learn new tasks.
- The goal is to find a parameter initialization θ that can be adapted to any new task using only a few training examples. This is why MAML is often called “learning to learn”.

MAML + CLIP FOR ZERO-SHOT LEARNING

TRAINING PHASE



TEST PHASE



Praveen Kumar Chandaliya, PhD
pkc@aid.svnit.ac.in
Department of Artificial Intelligence
Sardar Vallabhbhai National Institute of Technology , Surat

